

## *Multi – level Linked List*

*–Print List(Main List and Child)*

*–Time Complexity Calculation using Substitution Method*

**Program:**

```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "    ";
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }
        node = node->next;
    }
}

...

case 9:
    cout << "Current List Structure: " << endl;
    printList(head);
    break;
```

## *Imbalanced (Flat)*

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N - 1) + c & \text{for } n > 0 \end{cases}$$

$$T(N) = T(N - 1) + c$$

$$\therefore T(N - 1) = T(N - 2) + c$$

*Substituting in  $T(N) = T(N - 1) + c$ , we get:*

$$\Rightarrow T(N) = T(N - 2) + c + c \Rightarrow T(N - 2) + 2c$$

$$\therefore T(N - 2) = T(N - 3) + c$$

*Substituting in  $T(N) = T(N - 2) + 2c$ , we get:*

$$\Rightarrow T(N) = T(N - 3) + 2c + c \Rightarrow T(N - 3) + 3c$$

*if it goes for `i` time , we get:*

$$\Rightarrow T(N) = T(N - i) + ic$$

*As , we see  $T(0) = 1$  , i. e. Base Case when  $T(0)$ .*

$$\therefore N - i = 0 \text{ [0 for } T(0)]$$

$$\therefore i = N$$

$$\Rightarrow T(N) = T(N - N) + N \cdot c$$

$$\Rightarrow T(N) = T(0) + N \cdot c$$

*As,  $T(0)$  is constant, we can write it as:  $T(0) = c_0$*

$$\Rightarrow T(N) = c_0 + N \cdot c$$

*By applying Big - O, removing all constants we get:*

$$\Rightarrow T(N) = O(N)$$

---

*Alternatively,*

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N - 1) + c & \text{for } n > 0 \end{cases}$$

*This is a generalised version :*

$$T(N - 1) + c \quad \text{for } n > 0$$

*But actual equation:*

*The printList function contains a for loop that runs based on the current level:*

```
for (int i = 0; i < level; i++)  
    cout << "    ";
```

*This means as the recursive tree gets deeper, the work per node increases.*

*Let's define the new cost:*

- *Let  $c_2$  be the constant work at the node (checking nullptr, printing node  $\rightarrow$  data, etc.).*
- *Let  $c_1$  be the cost of printing a single space in the loop.*
- *Because level corresponds to the tree's depth  $k$ , the loop runs  $k$  times.*

*Therefore, Cost Per Problem at level `k` =  $c_1 \cdot k + c_2$ .*

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N-1) + c_1 \cdot k + c_2 & \text{for } n > 0 \end{cases}$$

$$T(n) = T(N-1) + c_1 \times k + c_2$$

$$T(N-1) = T(N-2) + c_1 k + c_2$$

***Hence , substituting this we get:***

$$T(n) = (T(N-2) + c_1 k + c_2) + c_1 k + c_2$$

$$\Rightarrow T(n) = T(N-2) + 2c_1 k + 2c_2$$

$$\Rightarrow T(n) = T(N-2) + 2(c_1 k + 2c_2)$$

$$T(N-2) = T(N-3) + c_1 k + c_2$$

***Hence , substituting this we get:***

$$\Rightarrow T(n) = T(N-3) + 3(c_1 k + 2c_2)$$

***If it goes for `i` times we get:***

$$\Rightarrow T(n) = T(N-i) + i(c_1 k + 2c_2)$$

***As base case is T(0), we get:***

$$N - i = 0, \text{ or, } i = N$$

$$\Rightarrow T(n) = T(N-N) + N(c_1 k + 2c_2)$$

$$\Rightarrow T(n) = T(0) + N(c_1k + 2c_2)$$

*We know ,  $T(0) = 1$  , therefore get:*

$$\Rightarrow T(n) = 1 + N(c_1k + 2c_2)$$

*By applying Big – O , removing all constants we get:*

$$\Rightarrow T(N) = O(N)$$

---

## ***Imbalanced (Deep)***

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N-1) + c \times N & \text{for } n > 0 \end{cases}$$

$$= T(N-1) + C \times N$$

$$T(N-1) = T(N-2) + c(N-1)$$

***Substituting this for T(N) we get:***

$$T(N) = T(N-2) + c(N-1) + cN$$

$$T(N-2) = T(N-3) + c(N-2)$$

***Substituting this for T(N) we get:***

$$T(N) = T(N-3) + c(N-2) + c(N-1) + cN$$

***If this goes upto level `k`, we get:***

$$T(N) = T(N-k) + c(n - (k-1)) + \dots + c(N-2) + c(N-1) + cN$$

***We can write it as:***

$$T(N) = T(N - k) + c \sum_{i=0}^{k-1} (n - i)$$

***As,  $T(0)$ , hence  $n - k = 0$  or  $k = n$ , therefore:***

$$T(N) = T(N - N) + c \sum_{i=0}^{n-1} (n - i)$$

$$\Rightarrow T(N) = T(0) + c \left( \begin{array}{c} (n - 0) + (n - 1) + (n - 2) + \dots \\ + n - (n - 2) + n - (n - 1) \end{array} \right)$$

$$\Rightarrow T(N) = T(0) + c \left( \begin{array}{c} n + (n - 1) + (n - 2) + \dots \\ + 2 + 1 \end{array} \right)$$

$$\Rightarrow T(N) = T(0) + c \left( \sum_{i=1}^n i \right)$$

***We know  $T(0) = 1$ , hence:***

$$\Rightarrow T(N) = 1 + c \left( \frac{n(n + 1)}{2} \right)$$

$$\Rightarrow T(N) = 1 + \frac{c}{2} (n(n + 1))$$



*Applying Big O we get:*

$$\Rightarrow T(N) = O(n(n + 1))$$

$$\Rightarrow T(N) = O(n^2 + n)$$

$$\Rightarrow T(N) = O(n^2)$$

---

*Alternatively,*

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N - 1) + c \times N & \text{for } n > 0 \end{cases}$$

*In general :*

*The printList function contains a for loop that runs based on the current level:*

```
for (int i = 0; i < level; i++)  
    cout << "    ";
```

*This means as the recursive tree gets deeper, the work per node increases.*

*Let's define the new cost:*

- *Let  $c_2$  be the constant work at the node (checking nullptr, printing node  $\rightarrow$  data, etc.).*
- *Let  $c_1$  be the cost of printing a single space in the loop.*

- *Because level corresponds to the tree's depth  $k$ , the loop runs  $k$  times.*

*Therefore, Cost Per Problem at level `k` =  $c_1 \cdot k + c_2$ .*

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N - 1) + c_1 k + c_2 & \text{for } n > 0 \end{cases}$$

*Here `k` will be  $N - 1$  because:*

*because at the top call the level is 0, at the next call the level is 1, ..., and when the problem size is  $N$ , the current level is  $N - (\text{current size})$ .*

$$T(n) = \begin{cases} 1 & \text{for } n = 0 \\ T(N - 1) + c_1 (N - 1) + c_2 & \text{for } n > 0 \end{cases}$$

$$T(N) = T(N - 1) + c_1(N - 1) + c_2$$

$$T(N - 1) = T(N - 2) + c_1(N - 2) + c_2$$

*Substitute  $T(N - 1)$ :*

$$T(N) = [T(N - 2) + c_1(N - 2) + c_2] + c_1(N - 1) + c_2$$

$$T(N) = T(N - 2) + c_1((N - 2) + (N - 1)) + 2c_2$$

*Again,*

$$T(N - 2) = T(N - 3) + c_1(N - 3) + c_2$$

*Substitute  $T(N - 2)$ :*

$$T(N) = (T(N - 3) + c_1(N - 3) + c_2) + c_1((N - 2) + (N - 1)) + 2c_2$$

$$\Rightarrow T(N) = T(N - 3) + c_1((N - 3) + (N - 2) + (N - 1)) + 3c_2$$

*if it goes upto `k` times we get:*

$$\Rightarrow T(N) = T(N - k) + c_1((N - k) + \dots + (N - 3) + (N - 2) + (N - 1)) + kc_2$$

*As ,  $T(0)$  is the base case , hence ,  $n - k = 0 \therefore n = k$ .*

$$\Rightarrow T(N) = T(N - N) + c_1((N - N) + \dots + (N - 3) + (N - 2) + (N - 1)) + Nc_2$$

$$\Rightarrow T(N) = T(N - N) + c_1(0 + \dots + (N - 3) + (N - 2) + (N - 1)) + Nc_2$$

$$\Rightarrow T(N) = T(0) + c_1\left(\sum_{i=0}^{n-1} i\right) + Nc_2$$

*We can write  $\sum_{i=0}^{n-1} i$  as  $\sum_{i=1}^{n-1} i$  as :  $a_0 + \sum_{i=1}^{n-1} i = 0 + \sum_{i=1}^{n-1} i = \sum_{i=1}^{n-1} i$*

$$\Rightarrow T(N) = T(0) + c_1 \left( \sum_{i=1}^{n-1} i \right) + Nc_2$$

*We know,*  $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$

$$\Rightarrow T(N) = T(0) + c_1 \times \frac{N(N-1)}{2} + Nc_2$$

*As , we know ,  $T(0) = 1$  , hence,*

$$\Rightarrow T(N) = 1 + \frac{N^2 - N}{2} c_1 + Nc_2$$

*Hence , applying Big – O , we get:*

$$\Rightarrow T(N) = O(n^2)$$

*as, constants  $c_1, c_2, \frac{1}{2}$  and 1 are negligible and  $N^2 > N$  , therefore,*

$$\Rightarrow T(N) = O(n^2)$$


---

**[Balanced Structure] – By Substitution Method.**

$$2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$

$$2T\left(\frac{N-1}{2}\right) + C, \text{ can also be written as : } 2T\left(\frac{N-(2-1)}{2}\right) + C$$

$$\begin{aligned} \text{Now, } T\left(\frac{N-1}{2}\right) &= 2T\left(\frac{\frac{N-1}{2}-1}{\frac{2}{1}}\right) + C = 2T\left(\frac{\frac{N-1-2}{2}}{\frac{2}{1}}\right) + C \\ &= 2T\left(\frac{N-3}{2} \times \frac{1}{2}\right) + C \\ &= 2T\left(\frac{N-3}{4}\right) + C \end{aligned}$$

$$\text{Substituting it in : } 2T\left(\frac{N-1}{2}\right) + C$$

$$\Rightarrow 2\left[2T\left(\frac{N-3}{4}\right) + C\right] + C$$

$$\Rightarrow 2^2T\left(\frac{N-3}{2^2}\right) + 2C + C$$

$$\Rightarrow 2^2T\left(\frac{N-(2^2-1)}{2^2}\right) + 2C + C$$

$$\begin{aligned}
 \text{Now, } T\left(\frac{N-3}{4}\right) &= 2T\left(\frac{\frac{N-3}{4}-1}{\frac{2}{1}}\right) + C = 2T\left(\frac{\frac{N-4-3}{4}}{\frac{2}{1}}\right) + C \\
 &= 2T\left(\frac{N-7}{8}\right) + C
 \end{aligned}$$

$$\text{Substituting it in : } 2^2 T\left(\frac{N-3}{4}\right) + 2C + C$$

$$\Rightarrow 2^2 \left[ 2T\left(\frac{N-7}{8}\right) + C \right] + 2C + C$$

$$\Rightarrow 2^3 T\left(\frac{N-7}{2^3}\right) + 2^2 C + 2C + C$$

$$\Rightarrow 2^3 T\left(\frac{N - (2^3 - 1)}{2^3}\right) + 2^2 C + 2C + C$$

***Thus ,if it runs upto `k` times we get:***

$$\Rightarrow 2^k T\left(\frac{N - (2^k - 1)}{2^k}\right) + 2^{k-1} C + \dots + 2^2 C + 2C + C$$

$$\Rightarrow 2^k T\left(\frac{N - (2^k - 1)}{2^k}\right) + C(2^{k-1} + \dots + 2^2 + 2^1 + 2^0)$$

Now, here  $2^{k-1} + \dots + 2^2 + 2^1 + 2^0$  is geometric progression.

Where number of terms counting from  $2^0, 2^1, \dots, 2^{k-1} = k$  terms

We can also find it out by:

Number of integers from  $a$  to  $b = b - a + 1$ .

Where  $b$  is ending index.

$a$  is starting index.

Here,  $b = k - 1$  and  $a = 0$ .

$(k - 1) - 0 + 1 = k$  terms.

By formula :  $S_n = a \left(\frac{r^n - 1}{r - 1}\right)$ , where  $a =$  starting value :  $2^0 = 1$

$r = 2$

$n =$  number of terms  $= k$

$$S_n = 2^0 \left(\frac{2^k - 1}{2 - 1}\right) = 1 \left(\frac{2^k - 1}{1}\right) = 2^k - 1.$$

$$\therefore 2^k T\left(\frac{N - (2^k - 1)}{2^k}\right) + C(2^k - 1)$$

As, we know,  $T(0) = 1$ , i. e. base case at 0 then:

$$\Rightarrow \frac{N - (2^k - 1)}{2^k} = 0$$

$$\Rightarrow N - (2^k - 1) = 0$$

$$\Rightarrow N - 2^k + 1 = 0$$

$$\Rightarrow N + 1 = 2^k$$

*Applying ,  $2^k = N + 1$  in formula :  $2^k T\left(\frac{N - (2^k - 1)}{2^k}\right) + C(2^k - 1)$ ,*

*We get:*

$$\Rightarrow (N + 1) \times T\left(\frac{N - ((N + 1) - 1)}{N + 1}\right) + C((N + 1) - 1)$$

$$\Rightarrow (N + 1) \times T\left(\frac{N - N}{N + 1}\right) + C(N)$$

$$\Rightarrow (N + 1) \times T\left(\frac{0}{N + 1}\right) + C(N)$$

$$\Rightarrow (N + 1) \times T(0) + C(N) \left[ \begin{array}{l} \text{since , } \frac{0}{a} = 0, \text{ where } a > 0 \\ \text{As } N + 1, \therefore N \geq 0. \end{array} \right]$$

*As,  $T(0)$  is constant , let  $T(0) = c_0$  , knowing  $T(0) = 1$  , and we can implement `1` instead of  $T(0)$  and end the process and we can also replace `C` with 1 ( as it is a standard )but lets keep it  $c_0$  and continue:*



$$\Rightarrow (N + 1) \times c_0 + c \times N [C \text{ or } c \text{ represents same constants}]$$

***By applying Big – O ,removing all constants we get:***

$$\Rightarrow O((N + 1)) + O(N)$$

$$\Rightarrow O(N)$$

**Note:**

***The recurrence:  $2T\left(\frac{N-1}{2}\right) + C$***

***arises from a balanced recursive structure of the multilevel linked list, but the algorithm itself is fundamentally recursive traversal rather than true divide – and – conquer.***

***Actual algorithm, it is NOT truly divide – and – conquer in the algorithmic sense.***

***The recurrence appears only because of a possible balanced recursion shape.***

***Hence, Real algorithm of Destroy List is NOT true divide – and – conquer.***

***$\therefore 2T\left(\frac{N-1}{2}\right) + C$  , this one is hypothetically assumed based on balanced recursive structure.***

---

***But this is not true :***

***As,***

***printList function contains a for loop that runs based on the current level:***

```
for (int i = 0; i < level; i++)  
    cout << "    ";
```

***This means as the recursive tree gets deeper, the work per node increases.***

***Let's define the new cost:***

- ***Let  $c_2$  be the constant work at the node (checking nullptr, printing node → data, etc.).***

```
while (node != nullptr)  
if (node->child != nullptr)  
cout << "|-- " << node->data << endl;  
node = node->next;  
int i = 0 ; i < level
```

***And call stack push on :***

```
printList(node->child, level + 1);
```

***With adding : Level + 1.***

- ***Let  $c_1$  be the cost of printing a single space in the loop.***

```
cout << "    ";
```

- ***Because level corresponds to the tree's depth  $k$ , the loop runs  $k$  times.***

*Therefore, Cost Per Problem at level  $k = c_1 \cdot k + c_2$ .*

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{ Base Case: } T(0) = 1.$$

*Hence, actual structure here is :*

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{ Base Case: } T(0) = 1.$$

*Hence by substitution method:*

$$T(n) = 2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2$$

$$T\left(\frac{N-1}{2}\right) = 2T\left(\frac{\frac{N-1}{2}-1}{2}\right) + c_1 \times k + c_2$$

$$= 2T\left(\frac{\frac{N-1-2}{2}}{2}\right) + c_1 \times k + c_2$$

$$= 2T\left(\frac{N-3}{4}\right) + c_1 \times k + c_2$$

*Hence, substituting we get:*

$$T(n) = 2\left(2T\left(\frac{N-3}{4}\right) + c_1 k + c_2\right) + c_1 k + c_2$$

$$= 2^2 T\left(\frac{N-3}{4}\right) + 2c_1 k + 2c_2 + c_1 k + c_2$$

$$= 2^2 T\left(\frac{N-3}{4}\right) + (2+1)(c_1 k + c_2)$$

*Again,*

$$T\left(\frac{N-3}{4}\right) = 2T\left(\frac{\frac{N-3}{4}-1}{2}\right) + c_1 \times k + c_2$$

$$= 2T\left(\frac{\frac{N-3-4}{4}}{2}\right) + c_1 \times k + c_2$$

$$= 2T\left(\frac{N-7}{8}\right) + c_1 \times k + c_2$$

*Hence , substituting we get:*

$$T(n) = 2^2 \left( 2T\left(\frac{N-7}{8}\right) + c_1 k + c_2 \right) + (2+1)(c_1 k + c_2)$$

$$= 2^3 T\left(\frac{N-7}{8}\right) + 2^2 c_1 k + 2^2 c_2 + (2+1)(c_1 k + c_2)$$

$$= 2^3 T\left(\frac{N-7}{8}\right) + (2^2 + 2 + 1)(c_1 k + c_2)$$

*After i<sup>th</sup> expansion , we get:*

$$T(n) = 2^i T\left(\frac{N - (2^i - 1)}{2^i}\right) + (2^{i-1} + \dots + 2^2 + 2 + 1)(c_1 k + c_2)$$

$$T(n) = 2^i T\left(\frac{N - (2^i - 1)}{2^i}\right) + \left(\sum_{i=0}^{i-1} 2^i\right)(c_1 k + c_2)$$

$$\sum_{i=0}^{i-1} 2^i, \text{ result to:}$$

$$S(n) = \sum_{i=0}^{i-1} 2^i = 2^0 + 2^1 + \dots + 2^{i-2} + 2^{i-1} = 1 + 2 + \dots + 2^{i-2} + 2^{i-1}$$

**Multiplying with 2 we get :**

$$2S(n) = 2^1 + 2^2 + \dots + 2^{i-2+1} + 2^{i-1+1} = 2 + 2^2 + \dots + 2^{i-1} + 2^i$$

$$2S(n) - S(n) = (0 - 1) + (2 - 2) + (2^2 - 2^2) + \dots + (2^{i-1} - 2^{i-1}) + (2^i - 0)$$

$$S(n) = 2^i - 1$$

$$\left[ \begin{array}{l} \text{Note : if there is } \sum_{i=0}^{i-1} 3^i, \text{ then we should have multiplied with 3 with } S \text{ i.e. } 3S \\ S = \sum_{i=0}^{i-1} r^i, \text{ then " } rS = r + r^2 + \dots + r^i \text{ ", where } r \text{ is the common ratio.} \\ rS - S = r^i - 1, \Rightarrow (r - 1)S = r^i - 1 \Rightarrow S = \frac{r^i - 1}{r - 1}, (r \neq 1) \end{array} \right]$$

$$\therefore T(n) = 2^i T\left(\frac{N - (2^i - 1)}{2^i}\right) + (2^i - 1)(c_1 k + c_2)$$

**we know,  $T(0)$  is our base case .**

$$\frac{N - (2^i - 1)}{2^i} = 0$$

***Multiplying  $2^i$  on both the sides:***

$$\Rightarrow N - (2^i - 1) = 0$$

***Moving the  $2^i - 1$  to the other side, we get:***

$$\Rightarrow N = 2^i - 1$$

***Add 1 to both sides to isolate the exponential term:***

$$\Rightarrow N + 1 = 2^i$$

***Take the base – 2 logarithm of both sides.***

***Because the base of our exponent is 2(from  $2^i$ ), we use the base – 2 logarithm ( $\log_2$ ) to target it.***

$$\Rightarrow \log_2(N + 1) = \log_2 2^i$$

$$\Rightarrow \log_2(N + 1) = i \times \log_2 2$$

$$\Rightarrow \log_2(N + 1) = i [\log_a a = 1]$$

$$\therefore i = \log_2(N + 1)$$

$$\therefore T(n) = 2^{\log_2(N+1)} T\left(\frac{N - (2^{\log_2(N+1)} - 1)}{2^{\log_2(N+1)}}\right) + (2^{\log_2(N+1)} - 1)(c_1 k + c_2)$$

$$\therefore T(n) = (N + 1) T\left(\frac{N - ((N + 1) - 1)}{N + 1}\right) + ((N + 1) - 1)(c_1 k + c_2)$$

$$\Rightarrow T(n) = (N + 1) T(0) + (N)(c_1 k + c_2)$$

***We know,  $T(0) = 1$***

$$\Rightarrow T(n) = (N + 1) \times 1 + (N)(c_1 k + c_2)$$

$$\Rightarrow T(n) = (N + 1) + (N)(c_1 k + c_2)$$

$$\Rightarrow T(n) = N + 1 + N c_1 k + N c_2$$

$$\Rightarrow T(n) = N(c_1 k + c_2 + 1) + 1$$

***If we observe recurrence tree ,  $i = k$  as here `i`  
is the level and for level `i` :  $c_1 \times i + c_2$***

***Similarly for level  $k$  :  $c_1 \times k + c_2$***

***Hence ,  $i = k$  . , therefore  $i = k = \log_2(N + 1)$***

**Hence,**

$$\Rightarrow T(n) = N(c_1(\log_2(N + 1)) + c_2 + 1) + 1$$

$$\Rightarrow T(n) = c_1 N \log_2(N + 1) + N c_2 + N + 1$$

$$\Rightarrow T(n) = \Theta(N \log_2 N)$$

$$2T\left(\frac{N-1}{2}\right) + C, \text{Base Case: } T(0) = 1.$$
$$= \Theta(N)$$

$\neq$

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{Base Case: } T(0) = 1.$$
$$= \Theta(N \log_2 N)$$

**$\therefore$  We will follow strictly:**

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{Base Case: } T(0) = 1.$$

$$= \Theta(N \log_2 N) \text{ is the correct answer.}$$



*And,*

$$2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$
$$= \Theta(N)$$



---